

Cloud Networking Deep Dive: Troubleshooting Real-Time Issues in AWS

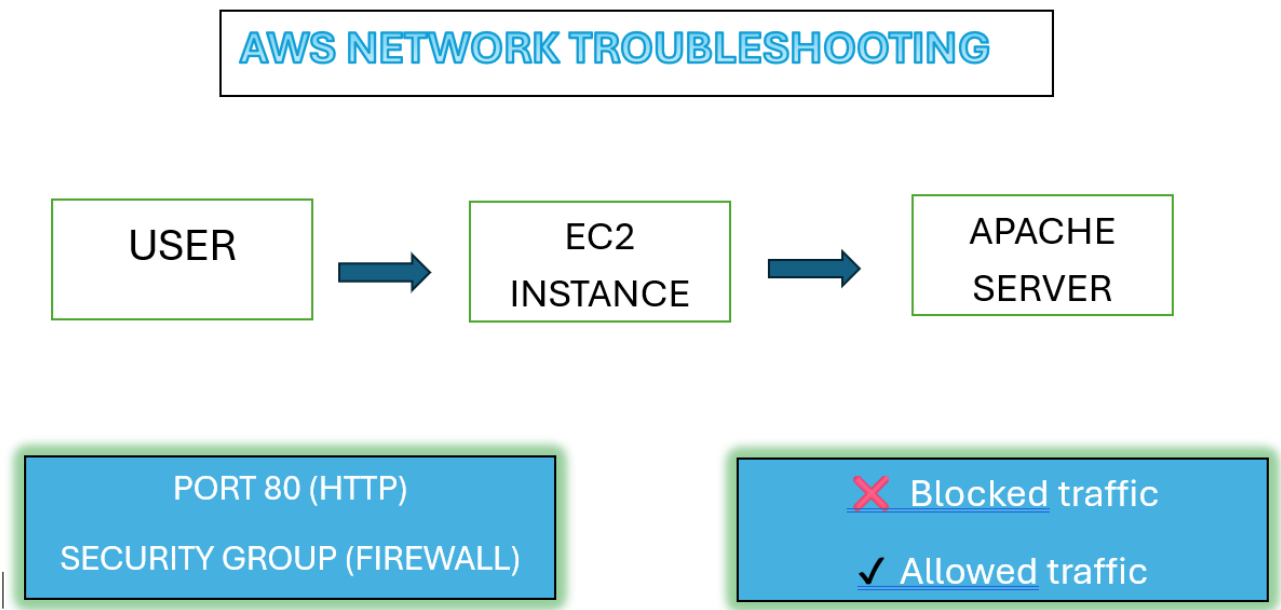
Overview

This project demonstrates practical understanding of networking concepts by simulating real-world network issues and resolving them using Linux and AWS tools.

Objectives

- Understand network communication in cloud
- Diagnose connectivity issues
- Work with ports, services, and firewalls
- Apply real troubleshooting techniques

Architecture Diagram



Setup Steps

- 1. Launch EC2 instance
- 2. Configure security group (SSH, HTTP)
- 3. Install Apache server
- 4. Verify access via public IP

Troubleshooting Scenarios

- Launched an ec2 instance with 2 inbound security group rules of ssh and http

z, A-Z, 0-9, spaces, and _-:/()#,@!+=&{}~`"

Description - *required* | [Info](#)

launch-wizard-7 created 2026-03-26T05:59:24.718Z

Inbound Security Group Rules

▼ Security group rule 1 (TCP, 22, 0.0.0.0/0)

Type | [Info](#)

ssh ▼

Protocol | [Info](#)

TCP

Port range | [Info](#)

22

Source type | [Info](#)

Anywhere ▼

Source | [Info](#)

Q Add CIDR, prefix list or security group

0.0.0.0/0 ✕

Description - *optional* | [Info](#)

e.g. SSH for admin desktop

Remove

▼ Security group rule 2 (TCP, 80, 0.0.0.0/0)

Type | [Info](#)

HTTP ▼

Protocol | [Info](#)

TCP

Port range | [Info](#)

80

Source type | [Info](#)

Anywhere ▼

Source | [Info](#)

Q Add CIDR, prefix list or security group

0.0.0.0/0 ✕

Description - *optional* | [Info](#)

e.g. SSH for admin desktop

Remove

The screenshot displays the AWS Management Console interface for an EC2 instance. At the top, there's a header for 'Instances (1/1)' with various action buttons like 'Connect', 'Instance state', 'Actions', and 'Launch instances'. Below this is a table listing instances, with the selected instance 'network-troubleshooting' (ID: i-0092991973fcde37c) in a 'Running' state. The instance details panel on the right shows various attributes:

- Instance ID:** i-0092991973fcde37c
- Public IPv4 address:** 16.171.238.15 | [open address](#)
- Private IPv4 addresses:** 172.31.31.21
- Instance state:** Running
- Public DNS:** ec2-16-171-238-15.eu-north-1.compute.amazonaws.com | [open address](#)
- Private IP DNS name (IPv4 only):** ip-172-31-31-21.eu-north-1.compute.internal
- Hostname type:** IP name: ip-172-31-31-21.eu-north-1.compute.internal

SCENARIO 1

Used Command and their purpose

Command : `sudo yum install httpd -y`

Usage : used to install Apache HTTP web server

The Apache HTTP Server is an open-source web server software used to process requests and serve web content over the internet. Installing it on your instance server provides a reliable, flexible, and widely supported platform for hosting websites and applications.

sudo (SuperUser DO): Allows a permitted user to execute a command with the privileges of the superuser (root). This is necessary because installing software modifies system-level files.

yum (Yellowdog Updater, Modified): The package manager used to install, update, and remove software packages in RPM-based Linux distributions. **install:** A subcommand for yum that tells it to download and install a new software package. **httpd:** The name of the Apache HTTP Server package in CentOS/RHEL repositories.

-y (Yes): An option (flag) that automatically answers "yes" to any confirmation prompts from the system during installation, such as verifying the download size or importing GPG keys.

```
[ec2-user@ip-172-31-31-21 ~]$ sudo yum install httpd -y
Amazon Linux 2023 Kernel Livepatch repository
Linux 2023 Kernel Livepatch repository
Dependencies resolved.

=====
Package                                Architecture      Version            Repository          Size
-----
Installing:
httpd                                  x86_64            2.4.66-1.amzn2023.0.1  amazonlinux          47 k
Installing dependencies:
apr                                    x86_64            1.7.5-1.amzn2023.0.4  amazonlinux          129 k
apr-util                              x86_64            1.6.3-1.amzn2023.0.2  amazonlinux          97 k
apr-util-ldap                         x86_64            1.6.3-1.amzn2023.0.2  amazonlinux          13 k
generic-logos-httpd                  noarch            18.0.0-12.amzn2023.0.3  amazonlinux          19 k
httpd-core                            x86_64            2.4.66-1.amzn2023.0.1  amazonlinux          1.4 M
httpd-filesystem                      noarch            2.4.66-1.amzn2023.0.1  amazonlinux          13 k
httpd-tools                           x86_64            2.4.66-1.amzn2023.0.1  amazonlinux          81 k
libbrotli                             x86_64            1.0.9-4.amzn2023.0.2  amazonlinux          315 k
mailcap                               noarch            2.1.49-3.amzn2023.0.3  amazonlinux          33 k
Installing weak dependencies:
apr-util-openssl                      x86_64            1.6.3-1.amzn2023.0.2  amazonlinux          15 k
mod_http2                             x86_64            2.0.27-1.amzn2023.0.3  amazonlinux          166 k
mod_lua                               x86_64            2.4.66-1.amzn2023.0.1  amazonlinux          60 k

Transaction Summary
-----
Install 13 Packages

Total download size: 2.4 M
Installed size: 6.9 M
Downloading Packages:
apr-1.7.5-1.amzn2023.0.4.x86_64.rpm    0% [=====]
7.5-1.amzn2023.0.4.x86_64.rpm          3.5 MB/s | 129 kB  00:00  --:--(1/13): apr-1.7.5-1.amzn2023.0.4.x86_64.rpm

i-0092991973fcde37c (network-troubleshooting)
PublicIPs: 16.171.238.15  PrivateIPs: 172.31.31.21
```

Command : `sudo systemctl start httpd`

Usage : to immediately start the Apache web server service (httpd) in the current session. The `systemctl` command is a powerful utility for managing the `systemd` system and service manager.

Command : `sudo systemctl enable httpd`

Usage : This subcommand tells `systemctl` to permanently enable the specified service which causes the service to be started when the system boots.

```
[ec2-user@ip-172-31-31-21 ~]$ sudo systemctl start httpd
[ec2-user@ip-172-31-31-21 ~]$ sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[ec2-user@ip-172-31-31-21 ~]$
```

i-0092991973fcde37c (network-troubleshooting)
PublicIPs: 16.171.238.15 PrivateIPs: 172.31.31.21

Outcome : The website works as the public IP address of the instance it opened in the browser



t works!

Command : `sudo systemctl stop httpd`

Usage : used in modern Linux systems to immediately stop the Apache web server service

```
ec2-user@ip-172-31-31-21 ~]$ sudo systemctl stop httpd
ec2-user@ip-172-31-31-21 ~]$
```

i-0092991973fcde37c (network-troubleshooting)
PublicIPs: 16.171.238.15 PrivateIPs: 172.31.31.21

Outcome : The website stops working



This site can't be reached

16.171.238.15 refused to connect.

Try:

- Checking the connection
- [Checking the proxy and the firewall](#)

ERR_CONNECTION_REFUSED

Reload

Details

Command : `sudo systemctl status httpd`

Usage : used on Linux systems (specifically RHEL-based distributions like CentOS and Fedora) to display the current operational status of the Apache web server service.

```
ec2-user@ip-172-31-31-21 ~]$ sudo systemctl stop httpd
ec2-user@ip-172-31-31-21 ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: inactive (dead) since Thu 2026-03-26 06:31:15 UTC; 2min 42s ago
     Duration: 6min 34.805s
    Docs: man:httd.service(8)
   Process: 26079 ExecStart=/usr/sbin/httpd $OPTIONS -DFOREGROUND (code=exited, status=0/SUCCESS)
   Main PID: 26079 (code=exited, status=0/SUCCESS)
    Status: "Total requests: 2; Idle/Busy workers 100/0; Requests/sec: 0.00514; Bytes served/sec: 4 B/sec"
      CPU: 428ms

Mar 26 06:24:39 ip-172-31-31-21.eu-north-1.compute.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
Mar 26 06:24:39 ip-172-31-31-21.eu-north-1.compute.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
Mar 26 06:24:39 ip-172-31-31-21.eu-north-1.compute.internal httpd[26079]: Server configured, listening on: port 80
Mar 26 06:31:14 ip-172-31-31-21.eu-north-1.compute.internal systemd[1]: Stopping httpd.service - The Apache HTTP Server...
Mar 26 06:31:15 ip-172-31-31-21.eu-north-1.compute.internal systemd[1]: httpd.service: Deactivated successfully.
Mar 26 06:31:15 ip-172-31-31-21.eu-north-1.compute.internal systemd[1]: Stopped httpd.service - The Apache HTTP Server.
ec2-user@ip-172-31-31-21 ~]$
```

i-0092991973fcde37c (network-troubleshooting)
PublicIPs: 16.171.238.15 PrivateIPs: 172.31.31.21

FIX : sudo systemctl start httpd command fixes this issue

```
ec2-user@ip-172-31-31-21 ~$ sudo systemctl start httpd
```

i-0092991973fcde37c (network-troubleshooting)

PublicIPs: 16.171.238.15 PrivateIPs: 172.31.31.21

← → ↻ ⚠ Not secure 16.171.238.15 ☆ 📄 🔍

It works!

SCENARIO 2

Removed the HTTP inbound rule in Security Group of the Instance

Removed HTTP

EC2 > Security Groups > sg-00a67e53d7b77e54e - troubleshooting-securitygroup > Edit inbound rules

Edit inbound rules Info

Inbound rules control the incoming traffic that's allowed to reach the instance.

Security group rule ID	Type Info	Protocol Info	Port range Info	Source Info	Description - optional Info	
sgr-01dfd9b040316b037	SSH	TCP	22	Custom		0.0.0.0/0 × Delete

Add rule

⚠ Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

×

Outcome : The website didn't work.

← → × ⓘ 16.171.238.15 ☆ 📄 🔍



This site can't be reached

16.171.238.15 took too long to respond.

Try:

- Checking the connection
- Checking the proxy and the firewall
- Running Windows Network Diagnostics

ERR_CONNECTION_TIMED_OUT

Reload

Details

FIX : used curl http://localhost command

Usage : used for transferring data to or from a server, supporting various protocols including HTTP, HTTPS, FTP, and SCP. It is widely used for downloading files, testing APIs, and automating network interactions.

curl = internal test this means Server is fine → Network is blocking

curl checks if the website is working inside the server curl worked ✓

→ server is fine

browser failed → problem is outside (network/security) curl didn't

fix anything — it helped to know where the problem is.

```
Mar 26 06:31:15 ip-172-31-31-21.eu-north-1.compute.internal systemd[1]: Stopped httpd.service - The Apache HTTP Server.
ec2-user@ip-172-31-31-21 ~]$ sudo systemctl start httpd
ec2-user@ip-172-31-31-21 ~]$ curl http://localhost
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
<title>It works! Apache httpd</title>
</head>
<body>
<p>It works!</p>
</body>
</html>
ec2-user@ip-172-31-31-21 ~]$
```

i-0092991973fcde37c (network-troubleshooting)

PublicIPs: 16.171.238.15 PrivateIPs: 172.31.31.21

FIX : ADDED HTTP AGAIN IN THE SECURITY GROUP OF THE INSTANCE

EC2 > Security Groups > sg-00a67e53d7b77e54e - troubleshooting-securitygroup > Edit inbound rules

Edit inbound rules

Inbound rules control the incoming traffic that's allowed to reach the instance.

Security group rule ID	Type	Protocol	Port range	Source	Description - optional	
sg-01dfd9b040316b037	SSH	TCP	22	Custom		Delete
-	HTTP	TCP	80	Anywh...	0.0.0.0/0 0.0.0.0/0 0.0.0.0/0	Delete

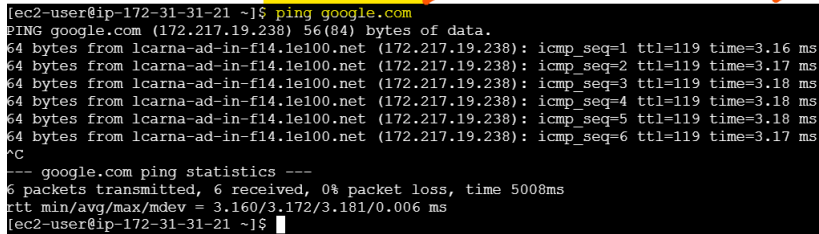
Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

Cancel Preview changes Save rules

added HTTP again

SCENARIO 3 (DNS TEST)

used **PING** command to test the reachability of the host on an Internet Protocol (IP) network and to measure the round-trip time (latency) for packets sent to that host. ping tests connectivity, Ctrl + C stops it



```
[ec2-user@ip-172-31-31-21 ~]$ ping google.com
PING google.com (172.217.19.238) 56(84) bytes of data:
64 bytes from lcarna-ad-in-f14.1e100.net (172.217.19.238): icmp_seq=1 ttl=119 time=3.16 ms
64 bytes from lcarna-ad-in-f14.1e100.net (172.217.19.238): icmp_seq=2 ttl=119 time=3.17 ms
64 bytes from lcarna-ad-in-f14.1e100.net (172.217.19.238): icmp_seq=3 ttl=119 time=3.18 ms
64 bytes from lcarna-ad-in-f14.1e100.net (172.217.19.238): icmp_seq=4 ttl=119 time=3.18 ms
64 bytes from lcarna-ad-in-f14.1e100.net (172.217.19.238): icmp_seq=5 ttl=119 time=3.18 ms
64 bytes from lcarna-ad-in-f14.1e100.net (172.217.19.238): icmp_seq=6 ttl=119 time=3.17 ms
^C
--- google.com ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5008ms
rtt min/avg/max/mdev = 3.160/3.172/3.181/0.006 ms
[ec2-user@ip-172-31-31-21 ~]$
```

i-0092991973fcd37c (network-troubleshooting)
PublicIPs: 16.171.238.5 PrivateIPs: 172.31.31.21

Ctrl + C - to stop PING

Used **nslookup** command to check if the domain name is correctly converting to an IP address as this command query the Domain Name System (DNS) to obtain domain names, IP address mappings, and other specific DNS records. It is primarily used for troubleshooting DNS-related problems and verifying domain configurations.

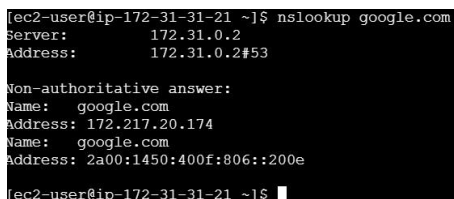
If you see an IP means:

- ✓ DNS is working
- ✓ Domain → IP conversion is successful

If it fails (no IP / error) means:

DNS problem

Server cannot resolve domain names



```
[ec2-user@ip-172-31-31-21 ~]$ nslookup google.com
Server:      172.31.0.2
Address:     172.31.0.2#53

Non-authoritative answer:
Name:   google.com
Address: 172.217.20.174
Name:   google.com
Address: 2a00:1450:400f:806::200e
[ec2-user@ip-172-31-31-21 ~]$
```

i-0092991973fcd37c (network-troubleshooting)
PublicIPs: 16.171.238.15 PrivateIPs: 172.31.31.21

Key Learnings

Difference between application vs network issues

- Importance of ports and firewall rules
- Structured debugging approach
- Real-world cloud troubleshooting mindset

Outcome

Gained hands-on experience in diagnosing and resolving real network issues in a cloud environment.